Phase 0 — Setup (Days 1–2)

- [ ] Download Gemma E4B `.litertlm` from the LiteRT Hugging Face community page — do this first because it is large and you want it on disk before you start coding
- [ ] Set up Flutter project with minimum Android SDK 26 (LiteRT-LM requirement — verify this in the docs)
- [ ] Set up your Android NDK version in `local.properties` — C++ FFI requires NDK, confirm the version LiteRT-LM C++ API was compiled against
- [ ] Clone the LiteRT-LM C++ guide repository and read the `inference_engine.h` header — understand what functions you will be calling before you write a single line of bridge code
- [ ] Register for Open-Meteo (no API key needed) and test a raw call for Bangalore coordinates returning humidity, temperature, UV index, and AQI
- [ ] Create your Flutter project structure: `lib/`, `android/app/src/main/cpp/`, `android/app/src/main/kotlin/` — you will be living in all three

---

Phase 1 — C++ FFI Bridge (Days 3–8)

This is your critical path. Do not touch UI until this works.

- [ ] Write the C++ wrapper (`litert_bridge.cpp`) that exposes three functions to Dart: `initModel(modelPath)`, `runInference(imageBytes, promptText)`, `freeModel()`. Keep the surface area minimal — complexity kills FFI bridges
- [ ] Write the `CMakeLists.txt` that links against LiteRT-LM's `.so` libraries — follow the C++ guide exactly here, do not improvise the linker flags
- [ ] Package the `.litertlm` file in `android/app/src/main/assets/` and write the Kotlin-side asset extraction logic that copies it to internal storage on first launch — LiteRT-LM needs a file path, not an asset stream
- [ ] Write the Dart FFI bindings in `lib/core/litert_ffi.dart` — define the `DynamicLibrary`, load it, and map each C function to a Dart typedef
- [ ] Write a minimal test: hardcode a single JPEG image and a hardcoded prompt, call `initModel` then `runInference`, print raw output to console. No UI, no database, just proof the bridge works
- [ ] Once the minimal test passes, add streaming support if LiteRT-LM exposes a token callback in its C++ API — streamed output makes the UI feel significantly more alive and is worth the extra complexity if supported
- [ ] Measure RAM usage during inference on a real mid-range Android device (target: under 3GB peak). If it exceeds this, you have a problem for low-end devices and need to document it honestly in your submission

---

Phase 2 — Prompt Architecture (Days 8–10)

This phase determines the quality of everything the user sees. Take it seriously.

- [ ] Write your system prompt. It needs to do four things: establish Gemma as a personal skin health monitor (not a diagnostic tool), tell it to reason over both the image and the structured environmental context together, tell it to speak in plain conversational English (not clinical language), and tell it to acknowledge when it has insufficient history to identify patterns. Draft this, test it, iterate
- [ ] Define your prompt template:

```
SYSTEM: {system_prompt}

USER: [IMAGE]
Today's environmental context for {city}:
- Humidity: {humidity}%
- Temperature: {temp}°C
- AQI: {aqi}
- UV Index: {uv}
- Time of day: {time}

My skin history summary (last 7 days): {history_summary}

Analyse my skin condition today and tell me what you observe
and what might be driving it based on the above context.
```

- [ ] Test this prompt with three different skin states: high oiliness, visible darkening, baseline normal. Evaluate: does Gemma actually reference the environmental data, or does it ignore it? If it ignores context, tighten the prompt
- [ ] Write the history summary generator — takes last 7 days of stored logs and compresses them into 3–4 sentences that fit within context budget without consuming too many tokens
- [ ] Define what a "cold start" response looks like on day 1 — Gemma should acknowledge no history exists and give general observational analysis only, not invented patterns

---

Phase 3 — Data Layer (Days 10–13)

- [ ] Add `isar` or `drift` to your Flutter project — `isar` is faster to set up for this use case
- [ ] Define schemas: `SkinEntry` (id, date, imagePath, gemmaInsight, oilinessRating self-reported optional, notes optional), `EnvironmentalEntry` (date, humidity, temp, aqi, uv, city), `UserProfile` (skinType, city, baselineToneReference, onboardingComplete)
- [ ] Write the image storage service — captures from camera, saves to app's private

directory, stores only the file path in the database. No image ever leaves the device

- [ ] Write the environmental data service — on each log entry, fetches current Open-Meteo data for the user's city and stores it alongside the skin entry
- [ ] Write the correlation query layer — pure Dart math, no ML needed. For each environmental variable, compute correlation against the implicit oiliness signal from Gemma's text output (or self-reported rating). Surface it as a human-readable finding when correlation coefficient exceeds 0.5 over at least 10 entries
- [ ] Write the weekly summary generator — aggregates the week's `SkinEntry` records into the history summary string format defined in Phase 2

Phase 4 — Flutter UI (Days 13–19)

- [ ] **Onboarding flow**: city picker (searchable, stores for Open-Meteo calls), skin type self-report (4 options with illustrations), Fitzpatrick scale selector (use illustrated swatches not numbers — this is important for Indian users unfamiliar with the clinical scale), camera permission request with a plain-language explanation of why and a reminder that photos never leave the device
- [ ] **Daily log screen**: camera capture with a centred oval face guide overlay, a single "Analyse" button, inference loading state with a progress indicator and estimated time (do not leave the user staring at a blank screen during a potentially 5–10 second inference), streamed Gemma response rendered word-by-word into a card
- [ ] **Timeline screen**: vertical calendar list, each day represented as a row with date, a colour-coded oiliness indicator, and a one-line excerpt from Gemma's insight. Tappable to expand full insight and the environmental context that was recorded that day
- [ ] **Correlations screen**: rendered only after 7+ entries exist. Cards that read like findings: "Your skin has been oilier on high humidity days (9 of 11 instances above 75% humidity)." Use `fl_chart` for a simple scatter plot overlay behind each finding
- [ ] **Weekly summary screen**: Gemma-generated, surfaced every Sunday, stored as a `SkinEntry` with a special type flag so it renders differently in the timeline
- [ ] **Privacy screen** (accessible from settings): one screen, plain language. "Every photo you take stays on your phone. We do not have a server. Your skin data is never transmitted anywhere." This is a feature, not legal boilerplate — design it like a feature

---

Phase 5 — Hardening (Days 19–22)

- [ ] Test the full flow on a real Android device, not an emulator — FFI bridges behave differently on real hardware
- [ ] Test on a low-end device if you can access one (3GB RAM) — document the result honestly
- [ ] Test image input edge cases: low light, face not centred, extreme angles. Add a pre

- [ ] Test image input edge cases: low light, face not centred, extreme angles. Add a pre-capture lighting check using the camera preview luminance — warn the user if conditions are too poor for reliable analysis
- [ ] Deliberately test with multiple skin tones — photograph different skin tone references and verify Gemma's responses are consistent and not biased toward lighter tones. Document this test in your submission
- [ ] Handle the inference failure case — if LiteRT-LM throws an exception (OOM, model load failure), surface a clear error with a retry option, never a silent crash
- [ ] Profile battery usage during inference — a single inference should not drain more than 2–3% battery. Document this

---

Phase 6 — Demo and Submission (Days 23–25)

- [ ] Script the demo as a narrative: open with the problem (you, Bangalore, skin changes you cannot explain), show onboarding, show three days of logs already populated, show the correlation card appearing, show Gemma's insight referencing both the image and the humidity data together, close with the privacy screen and the equity argument
- [ ] Record on a real device, good lighting, screen mirroring to your laptop for stable capture
- [ ] In the written submission, explicitly cover: why E4B on-device over cloud (privacy, equity for users without reliable data, offline capability), the Fitzpatrick gap in existing skin AI and how you addressed it in design, the C++ FFI bridge as a reusable open-source contribution other Flutter developers can use (this is a genuine community contribution worth highlighting), and honest limitations (RAM constraints on very low-end devices, inference latency)
- [ ] Open-source the FFI bridge as a standalone package on pub.dev or GitHub regardless of hackathon outcome — it does not exist yet and other developers will use it

---

The FFI bridge is where you will either win or lose time. Budget for it taking twice as long as you expect. Everything downstream of it is standard Flutter development which you already know from the BioClip2 project.